

Комплект 1: Структуры.

Задание 1.1

2. Постановка задачи.

Создать некоторую структуру с указателем на некоторую функцию в качестве поля. Вызвать эту функцию через имя переменной этой структуры и поле указателя на функцию.

3. Математическая модель.

$a + b = c;$

$a - b = c.$

4. Список идентификаторов.

Имя	Тип	Смысл
Schet	sstruct	Объявленная структура
sch	char	Символ операции
*func	int	Указатель на функцию
a	int	Переменная функции
b	int	Переменная функции
sum	int	Функция
sub	int	Функция
sch1	void	Переменная структуры
sch2	void	Переменная структуры
p_sch2	char	Указатель на функцию

5. Код программы.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Schet {
5      char sch;
6      int (*func) (int, int);
7  };
8
9  int sum(int a, int b) {
10     return a + b;
11 }
12
13 int sub(int a, int b) {
14     return a - b;
15 }
16
17 int main(void)
18 {
19     struct Schet sch1 = {'+', &sum};
20     struct Schet sch2 = {'-', &sub};
21     struct Schet *p_sch2 = &sch2;
22
23     int a = 2, b = 7;
24     printf("%d %c %d = %d\n", a, sch1.sch, b, sch1.func(a,b));
25     printf("%d %c %d = %d\n", a, p_sch2->sch, b, p_sch2->func(a,b));
26
27     return 0;
28 }
29

```

6. Результат выполненной работы.

```

2 + 7 = 9
2 - 7 = -5

Process returned 0 (0x0)   execution time : 1.103 s
Press any key to continue.

```

Задание 1.2

2. Постановка задачи.

Создать структуру для вектора в 3-х мерном пространстве. Реализовать и использовать в своей программе следующие операции над

векторами:

- скалярное умножение векторов;
- векторное произведение;

- модуль вектора;
- распечатка вектора в консоли.

В структуре вектора указать имя вектора в качестве отдельного поля этой структуры.

3. Математическая модель.

$$\vec{a} = (x_1, y_1, z_1);$$

$$\vec{b} = (x_2, y_2, z_2);$$

$$a \cdot b = x_1 * x_2 + y_1 * y_2 + z_1 * z_2$$

$$\vec{a} \times \vec{b} = \begin{vmatrix} i & j & k \\ x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \end{vmatrix}$$

$$|\vec{a}| = \sqrt{x_1^2 + y_1^2 + z_1^2}$$

4. Список идентификаторов.

Имя	Тип	Смысл
Vector	struct	Объявленная структура с координатами вектора
name	char	Символ структуры – имя вектора
x1	double	Координата вектора
x2	double	Координата вектора
x3	double	Координата вектора
skal_mult	double	Функция – скалярное произведение векторов
coordinates	double	Функция – векторное произведение
result	double	Переменная структуры
vlenght	double	Функция – длина вектора
print_vector	void	Функция, распечатывающая имя вектора и его координаты
skal_ab	double	Переменная, принимающая

		значение результата выполнения функции skal_mult
coord_ab	void	Функция, принимающая значение результата выполнения функции coordinates

5. Код программы.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  struct Vector {
6      char name[10];
7      double x1;
8      double x2;
9      double x3;
10 };
11
12 double skal_mult(struct Vector a, struct Vector b){
13     return a.x1 * b.x1 + a.x2 * b.x2 + a.x3 * b.x3;
14 };
15
16 struct Vector coordinates(struct Vector a, struct Vector b){
17     struct Vector result;
18     sprintf(result.name, "[%s, %s]", a.name, b.name);
19     result.x1 = a.x2 * b.x3 - a.x3 * b.x2;
20     result.x2 = a.x1 * b.x3 - a.x3 * b.x1;
21     result.x3 = a.x1 * b.x2 - a.x2 * b.x1;
22     return result;
23 };
24
25 double vlenght(struct Vector a){
26     return sqrt(a.x1 * a.x1 + a.x2 * a.x2 + a.x3 * a.x3);
27 };
28
29 void print_vector(struct Vector a){
30     printf("%s = {%.2f, %.2f, %.2f}\n", a.name, a.x1, a.x2, a.x3);
31 }

```

```

33 int main(void) {
34     struct Vector a = {"a", 1.1, 2.2, 3.3};
35     struct Vector b = {"b", 2.3, 3.7, 11.5};
36     print_vector(a);
37     print_vector(b);
38
39     double skal_ab = skal_mult(a, b);
40     printf("skal_mult = %.2f\n", skal_ab);
41
42     struct Vector coord_ab = coordinates(a, b);
43     print_vector(coord_ab);
44
45     printf("module a = %.2f\n", vlenght(a));
46     printf("module b = %.2f\n", vlenght(b));
47
48     return EXIT_SUCCESS;
49 }
50

```

6. Результат выполненной работы.

```

a = {1.10, 2.20, 3.30}
b = {2.30, 3.70, 11.50}
skal_mult = 25.47
{a, b} = {13.09, 5.06, -0.99}
module a = 4.12
module b = 12.30

Process returned 0 (0x0)   execution time : 0.188 s
Press any key to continue.

```

Задание 1.3

2. Постановка задачи.

Вычислить, используя структуру комплексного числа, комплексную экспоненту $\exp(z)$ некоторого $z \in \mathbb{C}$.

3. Математическая модель.

$$z = a + b \cdot i,$$

$$\exp(z) = e^{a+b \cdot i} = e^a * e^{bi} = e^a * (\cos(b) + i * \sin(b)) \quad - \text{ по формуле Эйлера.}$$

4. Список идентификаторов.

Имя	Тип	Смысл
complex	struct	Структура комплексного числа

de	double	Переменная (действительная часть комплексного числа)
mn	double	Переменная (мнимая часть числа)
a	double	Промежуточная переменная
print_complex_exp	void	Функция, распечатывающая результат (экспоненту комплексного числа)

5. Код программы.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  struct complex{
6      double de;
7      double mn;
8  };
9
10 void print_complex_exp(struct complex z){
11     double a = exp(z.de);
12     printf("exp(z) = %.6f + i * %.6f\n", a*cos(z.mn), a*sin(z.mn));
13 }
14
15 int main(void)
16 {
17     struct complex z = {1.5, 2.2};
18     print_complex_exp(z);
19
20     return 0;
21 }
22

```

6. Результат выполненной работы.

```

exp(z) = -2.637479 + i * 3.623429
Process returned 0 (0x0)   execution time : 0.070 s
Press any key to continue.

```

Задание 1.4

2. Постановка задачи.

Используя так называемые "битовые" поля в структуре C, создать экономную структуру в оперативной памяти для заполнения даты некоторого события, например, даты рождения человека.

3. Математическая модель.

$x.y.z, x \in [1, 31], y \in [1, 12], z \in [1, 4096]$

4. Список идентификаторов.

Имя	Тип	Смысл
DateOfBirth	struct	Объявленная структура
day	unsigned int	Переменная структуры
month	unsigned int	Переменная структуры
year	unsigned int	Переменная структуры

5. Код программы.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct DateOfBirth {
5      unsigned int day : 5;
6      unsigned int month : 4;
7      unsigned int year : 12;
8  };
9
10 int main(void) {
11
12     struct DateOfBirth Alex = {31, 5, 2022};
13     printf("DateOfBirth: %d.%d.%d", Alex.day, Alex.month, Alex.year);
14
15     return 0;
16 }
17
```

6. Результат выполненной работы.

```
DateOfBirth: 31.5.2022
Process returned 0 (0x0)   execution time : 0.391 s
Press any key to continue.
```

Задание 1.5

2. Постановка задачи.

Реализовать в виде структур двунаправленный связный список и совершить отдельно его обход в прямом и обратном направлениях с распечаткой значений каждого элемента списка.

3. Математическая модель.

-

4. Список идентификаторов.

Имя	Тип	Смысл
Node	struct	Структура списка
data	int	Переменная структуры
prev	int	Переменная структуры
next	int	Переменная структуры
push	void	Функция добавления нового элемента в начало списка
head_ref	int	Указатель на первый элемент списка
head	int	Указатель на первый элемент списка
tail	int	Указатель на последний элемент списка

5. Код программы.


```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node{
5      int data;
6      int prev;
7      int next;
8  };
9
10 typedef struct Node Node;
11
12 void push(Node** head_ref, int data){
13     Node* new_node = (Node*)malloc(sizeof(Node));
14
15     new_node->data = data;
16     new_node->prev = NULL;
17
18     new_node->next = (*head_ref);
19
20     if ((*head_ref) != NULL) {
21         (*head_ref)->prev = new_node;
22     }
23 }
```

```
24     (*head_ref) = new_node;
25 }
26
27 void print_list_forward(Node *node) {
28     while (node != NULL) {
29         printf("%d", node->data);
30         node = node->next;
31     }
32 }
33
34 void print_list_backwards(Node *node) {
35     while (node != NULL) {
36         printf("%d", node->data);
37         node = node->prev;
38     }
39 }
40
41 int main()
42 {
43     Node *head = NULL;
44     Node *tail = NULL;
45
46     push(&head, 3);
```

```

47     tail = head;
48
49     push(&head, 2);
50     push(&head, 1);
51
52     print_list_forward(head);
53     printf("\n");
54     print_list_backwards(tail);
55
56     return 0;
57 }
58

```

6. Результат выполненной работы.

```

123
321
Process returned 0 (0x0)   execution time : 0.168 s
Press any key to continue.

```

Комплект 2: Объединения и перечисления

Задание 2.1

2. Постановка задачи.

Напишите программу, которая использует указатель на некоторое объединение union.

3. Математическая модель.

-

4. Список идентификаторов.

Имя	Тип	Смысл
first	union	Входное объединение
num	int	Переменная объединения
sym	char	Переменная объединения
text	char	Переменная объединения
obj	union	Экземпляр объединения

p	union	Указатель на объединение
---	-------	--------------------------

5. Код программы.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  union first{
6      int num;
7      char sym;
8      char text[5];
9  };
10
11 int main(void)
12 {
13     union first obj;
14     obj.num = 16;
15
16     union first *p = &obj;
17
18     printf("Number: %d\n", p->num);
19
20     strcpy(p->text, "HAPPY");
21     printf("Text: %s", p->text);
22
23     return 0;
24 }
25

```

6. Результат выполненной работы.

```

Number: 16
Text: HAPPY
Process returned 0 (0x0)   execution time : 0.285 s
Press any key to continue.

```

Задание 2.2

2. Постановка задачи.

Напишите программу, которая использует union для побайтовой распечатки типа unsigned long.

3. Математическая модель.

-

4. Список идентификаторов.

Имя	Тип	Смысл
Un	void	Функция побайтового вывода числа
var	unsigned long	Переменная функция
varvar	unsigned long	Переменная объединения
A	unsigned char	Переменная объединения
number	union	Объединение

5. Код программы.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void Un (unsigned long var){
5      union {
6          unsigned long varvar;
7          unsigned char A[8];
8      } number;
9      number.varvar = var;
10     printf("Bytes: ");
11     for (int i=7; i>=0; i--)
12         printf(" %x", number.A[i]);
13 }
14
15 int main(void){
16
17     Un(1844674407);
18
19     return 0;
20 }
21
```

6. Результат выполненной работы.

```
Bytes:  0 0 0 0 6d f3 7f 67
Process returned 0 (0x0)   execution time : 0.137 s
Press any key to continue.
```

Задание 2.3

2. Постановка задачи.

Создайте перечислимый тип данных (enum) для семи дней недели и распечатайте на экране его значения, как целые числа.

3. Математическая модель.

-

4. Список идентификаторов.

Имя	Тип	Смысл
week	enum	Список дней недели
monday	int	Константа списка
tuesday	int	Константа списка
wednesday	int	Константа списка
thursday	int	Константа списка
friday	int	Константа списка
saturday	int	Константа списка
sunday	int	Константа списка

5. Код программы.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  enum week {monday = 1, tuesday, wednesday, thursday, friday, saturday, sunday};
5
6  int main()
7  {
8      printf("%d\n", monday);
9      printf("%d\n", tuesday);
10     printf("%d\n", wednesday);
11     printf("%d\n", thursday);
12     printf("%d\n", friday);
13     printf("%d\n", saturday);
14     printf("%d\n", sunday);
15     return 0;
16 }
17
```

6. Результат выполненной работы.

```
1
2
3
4
5
6
7
Process returned 0 (0x0)   execution time : 0.038 s
Press any key to continue.
```

Задание 2.4

2. Постановка задачи.

Создайте так называемое размеченное объединение union, которое заключено в виде поля структуры struct вместе с ещё одним полем, которое является перечислением enum и служит индикатором того, что именно на текущий момент хранится в таком вложенном объединении. Создать и заполнить динамический массив таких структур с объединениями внутри, заполняя вспомогательное поле перечисления enum для сохранения информации о хранимом в каждом размеченном объединении типе данных. Реализовать распечатку данных массива таких структур в консоль.

3. Математическая модель.

-

4. Список идентификаторов.

Имя	Тип	Смысл
Smart	struct	Входная структура
UnionType	union	Входное объединение
number	int	Переменная объединения
bytes	char[4]	Переменная объединения
UnionTypeState	enum	Список
NUMBER	int	Константа списка UnionTypeState
BYTES	int	Константа списка UnionTypeState
n	const	мощность массива A
A[n]	array of structs	Массив структур
i	int	Счётчик цикла

5. Код программы.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Smart {
5      union UnionType {
6          int number;
7          char bytes[4];
8      } union_object;
9      enum UnionTypeState {
10         NUMBER = 0, BYTES = 1
11     } state;
12 };
13
14 int main()
15 {
16     const n = 5;
17     struct Smart A[n];
18
19     A[1].union_object.number = 13;
20     A[1].state = NUMBER;
21
22     strcpy(A[2].union_object.bytes, "1d");
23     A[2].state = BYTES;
24
25     A[3].union_object.number = 2567;
26     A[3].state = NUMBER;
27
28     A[4].union_object.number = 0;
29
30     A[4].state = NUMBER;
31
32     strcpy(A[5].union_object.bytes, "3fd");
33     A[5].state = BYTES;
34
35     for (int i=1; i<6; i++){
36         if (A[i].state == 0)
37             printf("Type: %d (NUMBER), value: %d\n", A[i].state, A[i].union_object.number);
38         else
39             printf("Type: %d (BYTES), value: %s\n", A[i].state, A[i].union_object.bytes);
40     };
41
42     return 0;
43 }

```

6. Результат выполненной работы.

```

Type: 0 (NUMBER), value: 13
Type: 1 (BYTES), value: 1d
Type: 0 (NUMBER), value: 2567
Type: 0 (NUMBER), value: 0
Type: 1 (BYTES), value: 3fd

Process returned 0 (0x0)   execution time : 0.232 s
Press any key to continue.

```